

Pedra, Papel e Tesoura: *a primeira versão*

Um projeto novo, em três aulas: um jogo de Pedra, Papel e Tesoura contra o computador, em **Python + Streamlit**. Nesta primeira versão, o agente cria a regra do jogo num módulo separado, a tela com três botões e o resultado de cada rodada. E você aprende a pedir testes que cobrem **todas as combinações**.

Projeto novo

jogo.py + app.py

random.choice

Dicionário de regras

9 combinações

Teste com sorte controlada

Formação Claude Code 2026 · Baseado em cursor.com/docs (Agent, Plan Mode, Rules, Browser), docs.python.org (random, unittest.mock) e docs.streamlit.io, consultados em outubro de 2026 (Streamlit 1.65) · Código de referência testado em [projeto/](#) · Telas: print real

Um jogo, *três aulas*

AULA 13 · HOJE

Botões, jogada do computador e resultado



AULA 14

Imagens para as jogadas



AULA 15

Placar e histórico de rodadas

ESCOPO DA V1

- ✓ Três botões: Pedra, Papel e Tesoura
- ✓ O computador escolhe ao acaso, no mesmo clique
- ✓ Mostra as duas jogadas e o resultado: vitória, derrota ou empate
- ✓ Regra do jogo num arquivo separado: `jogo.py`

A REGRA

Pedra quebra a tesoura · **Papel** embrulha a pedra · **Tesoura** corta o papel.
Jogadas iguais empatam.

Mesmo ambiente do ToDo List: pasta nova, `.venv`, `pip install streamlit`, `git init` e uma rule do projeto (aula 10).

O pedido *da versão 1*

Crie um jogo de Pedra, Papel e Tesoura em Streamlit, contra o computador.

- jogo.py: as opções, quem vence quem, a jogada do computador (aleatória) e uma função resultado(jogador, computador) que devolve "vitoria", "derrota" ou "empate";
- app.py: três botões lado a lado; ao clicar, mostre a sua jogada, a do computador e o resultado (st.success, st.error ou st.info). Sem placar por enquanto. Crie também um teste das 9 combinações de jogadas.

DECISÃO DO PEDIDO	POR QUÊ
Regra em <code>jogo.py</code>	Dá para testar sem abrir o app
Retorno com três valores fixos	A tela decide a mensagem; a regra só decide quem ganha
"Sem placar por enquanto"	Segura o escopo (placar é da aula 15)
"Teste das 9 combinações"	3 × 3 jogadas: cobre tudo

Separar **regra** de **tela** é a decisão mais importante do projeto. Vale pedir no Plan e conferir antes do código.

A regra *em jogo.py*

```
"""Regras do Pedra, Papel e Tesoura."""
import random

OPCOES = ["Pedra", "Papel", "Tesoura"]
# Quem cada opção vence.
VENCE = {"Pedra": "Tesoura", "Papel": "Pedra", "Tesoura": "Papel"}

def jogada_do_computador():
    return random.choice(OPCOES)

def resultado(jogador, computador):
    """Devolve 'vitoria', 'derrota' ou 'empate' do ponto de vista do jogador."""
    if jogador == computador:
        return "empate"
    return "vitoria" if VENCE[jogador] == computador else "derrota"
```

- ✓ `VENCE` diz quem cada opção derrota: uma tabela no lugar de vários `if`
- ✓ `random.choice` sorteia uma das três opções
- ✓ `resultado()` não sabe nada de tela: só recebe duas jogadas e devolve uma palavra

Com a tabela `VENCE`, uma variante como "Pedra, Papel, Tesoura, Lagarto, Spock" vira só mais linhas no dicionário, com uma lista de quem cada um vence.

A tela *em app.py*

```
import streamlit as st

from jogo import OPCOES, jogada_do_computador, resultado

st.set_page_config(page_title="Pedra, Papel e Tesoura", page_icon="🎲")
st.title("🎲 🖐️ ✂️ Pedra, Papel e Tesoura")
st.write("Escolha sua jogada. O computador escolhe ao mesmo tempo, ao acaso.")

EMOJI = {"Pedra": "👊", "Papel": "🖐️", "Tesoura": "✂️"}

colunas = st.columns(3)
escolha = None
for coluna, opcao in zip(colunas, OPCOES):
    if coluna.button(f"{EMOJI[opcao]} {opcao}", width="stretch"):
        escolha = opcao

if escolha:
    computador = jogada_do_computador()
    st.write(f"Você: **{EMOJI[escolha]} {escolha}** · Computador: **{EMOJI[computador]} {computador}**")
    final = resultado(escolha, computador)
    if final == "vitoria":
        st.success("Você venceu! 🎉")
    elif final == "derrota":
        st.error("O computador venceu.")
    else:
        st.info("Empate!")
```

- ✓ `st.columns(3)` põe os botões lado a lado
- ✓ Botão do Streamlit devolve `True` só na execução do clique
- ✓ A jogada do computador é sorteada **depois** do clique, nunca antes
- ✓ Três cores de mensagem para os três resultados

Repare: ao clicar em qualquer outro lugar, o resultado **some**, porque o botão só vale naquela execução. A aula 14 resolve isso guardando a rodada na sessão.

A versão 1 *funcionando*



Print real do `projeto/` da aula: Pedra contra Tesoura

Para o print, a jogada do computador foi fixada em Tesoura. No jogo normal ela é sorteada a cada clique.

As 9 combinações

VOCÊ \ COMPUTADOR	PEDRA	PAPEL	TESOURA
Pedra	empate	derrota	vitória
Papel	vitória	empate	derrota
Tesoura	derrota	vitória	empate

O código de referência passou nas 9: a tabela acima é a saída do teste.

TESTE DA REGRA (PYTEST)

```
from itertools import product
from jogo import OPCOES, resultado

ESPERADO = {"Pedra", "Tesoura": "vitoria",
            ("Papel", "Pedra"): "vitoria",
            ("Tesoura", "Papel"): "vitoria"}

def test_todas_as_combinacoes():
    for j, c in product(OPCOES, repeat=2):
        if j == c:
            assert resultado(j, c) == "empate"
        elif (j, c) in ESPERADO:
            assert resultado(j, c) == "vitoria"
        else:
            assert resultado(j, c) == "derrota"
```

Rodar: `pip install pytest` e `pytest` . O agente pode criar e rodar isso para você.

Testando *o que é aleatório*

Como testar a tela se o computador joga ao acaso? **Controlando a sorte:** no teste, a escolha aleatória é substituída por um valor fixo.

```
from unittest import mock
from streamlit.testing.v1 import AppTest

def test_pedra_vence_tesoura():
    at = AppTest.from_file("app.py").run()
    with mock.patch("random.choice", return_value="Tesoura"):
        at.button[0].click().run()  # botão Pedra
    assert "venceu" in at.success[0].value
```

- ✓ `mock.patch` troca `random.choice` só dentro do `with`
- ✓ O `AppTest` clica no botão sem abrir o navegador
- ✓ O teste fica **repetível**: sempre o mesmo resultado

Peça assim ao agente: *"crie um teste da tela que fixa a jogada do computador com mock e confere a mensagem de vitória, derrota e empate"*.

Testes que dependem da sorte passam umas vezes e falham outras. Isso é pior do que não ter teste.

Jogando *pele agente*

@Browser Abra o jogo em localhost:8501 e jogue 10 rodadas, alternando Pedra, Papel e Tesoura. Para cada rodada, anote a sua jogada, a do computador e a mensagem. No fim, diga se alguma mensagem não bate com a regra do jogo.

Esse teste no navegador confere a **integração**: botão certo, jogada certa, mensagem certa. A regra em si já está coberta pelos testes da seção 06.

Dez rodadas também mostram se o sorteio parece justo. Com poucos jogos, sequências como "três Tesouras seguidas" são normais.

Checklist *e commit*

- ✓ Os três botões funcionam
- ✓ O computador escolhe ao acaso a cada clique
- ✓ As 9 combinações dão o resultado certo
- ✓ Regra em `jogo.py`, tela em `app.py`
- ✓ `requirements.txt` e `.gitignore` criados

```
git add .  
git commit -m "Pedra, Papel e Tesoura: primeira versão"
```

Ideias de ajuste para treinar: um botão "Jogar de novo", uma frase explicando por que você ganhou ("Pedra quebra tesoura").

Teste o que *ficou*

1. Por que a regra fica em jogo.py, separada da tela?

A) O Streamlit exige

B) Para testar a regra sem abrir o app

C) Para o app ficar mais rápido

D) Não há motivo

B. Regra pura é fácil de testar.

2. Quantas combinações de jogadas existem?

A) 3

B) 6

C) 9

D) 27

C. 3 opções × 3 opções.

3. Como testar a tela se a jogada do computador é aleatória?

A) Rodar muitas vezes e torcer

B) Fixar random.choice com mock.patch

C) Não dá para testar

D) Tirar o random do jogo

B. Teste repetível.

4. Por que o resultado some ao clicar em outro lugar?

A) Bug do Streamlit

B) O botão só vale na execução do clique

C) O navegador limpa a tela

D) Falta internet

B. A aula 14 guarda a rodada na sessão.

Regra separada, *jogo testado*

O jogo tem a regra em `jogo.py` (uma tabela `VENCE` e a função `resultado()`) e a tela em `app.py`. Testamos as **9 combinações** e a tela com a **sorte controlada** por `mock`. Na próxima aula, o jogo ganha imagens.

`jogo.py` · `app.py`

`random.choice`

Tabela `VENCE`

9 combinações

`mock.patch`

Próxima aula → 14 · Pedra, Papel e Tesoura · Adicionando Imagens no Jogo

Fontes: cursor.com/docs (Agent Overview, Plan Mode, Browser); docs.python.org (`random.choice`, `itertools.product`, `unittest.mock`); docs.streamlit.io (`st.button`, `st.columns`, `AppTest`), consultados em outubro de 2026. Código de referência em `projeto/`, testado com Streamlit 1.65.